COSC 6385: Advanced Computer Architecture

Programming Assignment: Branch Prediction

Pentium M Hybrid Branch Predictor (with Partial Extra Credit)

Git repo link: https://github.com/K-a-v-y-a-s-r-i/Branch-Predictor_KSP

Kavya Sri Puvvada

University of Houston

October 2026

1 Introduction

The goal of this assignment is to implement the branch outcome (taken / not taken) part of the Pentium M branch predictor described in "Experiment Flows and Microbenchmarks for Reverse Engineering of Branch Predictor Structures" (Uzelac and Milenković, ISPASS 2009), inside the CBP2 simulation infrastructure, and to evaluate it on the 20 provided SPEC CPU2000 traces.

Summary of what I did:

1. **Required part:** implemented `pm_predictor` in `src/my_predictor.h`, a hybrid of a **bimodal predictor** and a **tagged, set-associative global predictor**. The global prediction is used on a hit, otherwise the bimodal prediction. As allowed, the loop predictor, target prediction and the PIR/IR hash are not part of `pm_predictor`, and the global table is 2-way.
2. **Evaluation:** ran it on all 20 traces against a bimodal-only baseline and the provided 32K-entry gshare, plus a design-space study (ways, sets, history length).
3. **Extra credit (partial):** implemented `cpm_predictor`, an extended, *partial* model of the Pentium M predictor: bimodal + global + **loop predictor**, plus **branch target prediction** with a BTB. The indirect BTB and the PIR hash are not modelled (Section 5).
4. Found an issue in the provided framework that makes target prediction impossible to measure, and worked around it **without modifying any framework file** (Section 6). Only `src/my_predictor.h` was edited; everything else is new files.

Results: the submitted predictor averages **6.217 MPKI** (bimodal only 10.170, gshare 32K 6.306) using about a quarter of gshare's storage. The extended (partial) Pentium M predictor (extra credit) reaches **5.831 MPKI**.

2 Design of pm_predictor

2.1 Bimodal (local) predictor

- 4096 entries of 2-bit saturating counters (12 index bits).
- Indexed by the word address of the branch: `(address >> 2) & 0xFFF` (as in the hand example, which drops the 2 low address bits).
- Counters start at 2 (weakly taken) and are trained on every conditional branch.

2.2 Global predictor

- Organized like a cache: 512 sets × 2 ways. Each entry holds {valid, 6-bit tag, 2-bit counter, LRU age}.
- Global History Register (GHR): outcomes of the last 15 conditional branches.
- `h = (address >> 2) XOR GHR`: tag = low 6 bits of `h`, set (row) = next 9 bits, the same split as the hand example (row = upper bits, tag = lower bits of the XOR result).
- Lookup compares the tag with every way of the row. A valid matching entry is a **hit**.

2.3 Final prediction

- Global hit → prediction of the hit entry's counter.

- Global miss → bimodal prediction.

2.4 Update

1. The bimodal counter is always updated with the actual outcome.
2. Global hit: update that entry's counter and make it most-recently-used.
3. Global miss **and** the bimodal predictor was wrong: allocate an entry in that row (an invalid way first, otherwise the LRU way), store the tag, and set the counter to weakly taken (2) or weakly not taken (1) according to the outcome. Allocating only when the bimodal predictor fails keeps easy branches out of the global table, as in the Pentium M design.
4. Shift the outcome into the GHR.

The global table uses **true LRU** (an age counter per way), so the same code works for any associativity, including the real 4-way Pentium M table. All sizes are compile-time parameters (`PM_WAYS`, `PM_ROW_BITS`, `PM_TAG_BITS`, `PM_HIST_LEN`, `PM_BIM_BITS`; `-DPM_NO_GLOBAL` builds the bimodal-only baseline).

Storage: bimodal 4096 × 2 bits = 1 KB; global 1024 × (6 + 2 + 1 + 1) bits about 1.25 KB; total about 2.25 KB, compared with 8 KB for the provided gshare.

No external cache code was used: the set-associative tables were written from scratch.

3 Results

Direction MPKI (mispredictions per 1000 instructions, lower is better) on all 20 traces. The bimodal and gshare columns are baselines; **PM 2-way** is the submitted `pm_predictor`; **PM 4-way 2K** is the same code built with `-DPM_WAYS=4` (Pentium M-like 2048-entry global table); the last two columns are the extra-credit `cpm_predictor` (extended, partial Pentium M).

Trace	Bimodal 4K	gshare 32K	PM 2-way (submitted)	PM 4-way 2K	Extended PM: direction	Extended PM: target
164.gzip	15.029	12.473	12.673	12.606	11.294	11.913
175.vpr	14.629	13.415	13.813	13.398	13.541	14.077
176.gcc	19.136	11.254	10.674	9.367	10.188	13.337
181.mcf	33.234	15.837	16.776	15.895	16.670	17.133
186.crafty	12.713	5.837	5.986	5.310	5.878	6.271
197.parser	15.527	10.008	9.224	8.731	9.013	9.224
201.compress	9.193	7.831	7.822	7.678	7.520	7.541
202.jess	8.736	1.562	1.094	1.057	0.935	1.476
205.raytrace	3.462	2.756	2.522	2.377	2.229	4.427
209.db	6.302	3.909	3.662	3.611	3.508	5.023
213.javac	3.234	2.267	2.228	2.068	1.704	2.595
222.mpegaudio	3.201	2.188	2.213	2.056	1.692	2.501
227.mtrt	3.890	2.657	2.631	2.575	2.382	3.898
228.jack	6.665	3.033	2.498	2.447	2.292	3.697
252.eon	9.406	1.807	1.673	1.596	1.222	1.291
253.perlbmk	6.108	2.554	1.932	1.845	1.653	1.763
254.gap	8.862	3.926	3.994	3.834	3.844	4.761

Trace	Bimodal 4K	gshare 32K	PM 2-way (submitted)	PM 4-way 2K	Extended PM: direction	Extended PM: target
255.vortex	2.409	1.222	0.925	0.859	0.834	1.386
256.bzip2	0.113	0.094	0.063	0.062	0.045	0.051
300.twolf	21.552	21.489	21.932	21.687	20.180	20.516
Average	**10.170**	**6.306**	**6.217**	**5.953**	**5.831**	**6.644**

- Adding the global predictor to the bimodal table cuts the average MPKI from 10.170 to 6.217 (-39%).
- The submitted predictor beats the 32K-entry gshare on average (6.217 vs 6.306) and on 13 of 20 traces, with about a quarter of the storage.
- The extended Pentium M predictor is the best overall (5.831) and beats gshare on 17 of 20 traces.

4 Design-space exploration

Average MPKI over the 20 traces:

Configuration	Global entries	Average MPKI
Bimodal only (no global predictor)	0	10.170
2-way, 256 sets	512	6.582
2-way, 512 sets (submitted)	**1024**	**6.217**
4-way, 256 sets (same storage as submitted)	1024	6.321
4-way, 512 sets (Pentium M-like)	2048	5.953
2-way, 512 sets, 10-bit GHR	1024	7.578
Provided gshare	32768	6.306
Extended Pentium M (extra credit)	1024 + loop	5.831

- More global entries help (512 $\rightarrow$ 1024 $\rightarrow$ 2048: 6.582 $\rightarrow$ 6.217 $\rightarrow$ 5.953).
- For the same storage, 2-way with more sets (6.217) is slightly better than 4-way with fewer sets (6.321): these traces gain more from extra index bits than from extra associativity.
- History length matters: a 10-bit GHR gives 7.578, compared with 6.217 with 15 bits.

5 Extra credit: extended (partial) Pentium M predictor

The class `cpm_predictor` (run with `predict_extra_credit`, or `./run_extra.sh traces` for correct target numbers) implements both outcome and target prediction. It is a **partial** model of the Pentium M predictor, not a reproduction of every structure in the paper.

Not modelled:

- the indirect-branch BTB (iBTB);
- the path information register (PIR) and its hash with the IP: the global predictor is indexed with a global *outcome* history register XOR the address, as in `pm_predictor`;
- the return stack buffer;
- the paper's exact loop-predictor organisation: my loop predictor is direct-mapped with 14-bit tags and a usefulness filter, which is my own design.

The indirect BTB and the return stack would not change the reported numbers, because the provided driver only scores conditional branches.

Outcome prediction (priority: loop > global > bimodal):

- Bimodal + global predictor: the `pm_predictor` above is reused.
- **Loop predictor**: 128 entries, direct-mapped, 14-bit tags. Each entry learns the *trip count* of a loop branch (taken N times, then not taken once): `iter` counts the taken iterations of the current loop instance; at the not-taken exit, if `iter` equals the stored `trip` the confidence counter is incremented, otherwise `trip` is replaced and confidence reset. With confidence at least 3 the entry predicts *not taken* when `iter == trip` and *taken* otherwise, so it predicts loop exits that a 2-bit counter always gets wrong.
- An entry is allocated when a loop exit (not-taken branch) is mispredicted. A confident entry is aged instead of being evicted immediately.
- A per-entry **usefulness counter** tracks whether the loop prediction was right when it disagreed with the global/bimodal prediction. The loop predictor overrides only when it is useful. Without this filter the loop predictor made mcf and jess worse; with it, it helps on most traces.

Target prediction:

- **BTB**: 2048 entries, 4-way set-associative, true LRU, tagged. Each entry stores the taken target and the fall-through address of the branch.
- In this trace format the `target` of a branch is the actual next PC (the fall-through address when not taken), so the predicted target is the BTB taken target if the branch is predicted taken, otherwise the fall-through address.

Average results: **direction 5.831 MPKI** (vs 6.217 without the loop predictor) and **target 6.644 MPKI**. The target MPKI is only slightly higher than the direction MPKI: almost all target errors come from a wrong direction, and the BTB almost always has the correct address.

6 Issue found in the provided infrastructure

In the provided `src/predictor.h`, the getter `bool target_prediction ()` returns **bool**, while the stored value is an `unsigned int` address. Every predicted target is therefore truncated to 0 or 1, and `predict_extra_credit` counts almost every conditional branch as a target misprediction: about 141 target MPKI on jess, whatever the predictor.

So that the code builds and behaves the same with the original framework, **I did not modify** `predictor.h` **or any other framework file**. Instead:

- `cpm_update` (in `my_predictor.h`) keeps a public copy of the predicted target, `pred_target`;
- the new driver `src/measure_target.cc` is identical to `predict_extra_credit.cc`, except that it compares `pred_target` with the actual target. `./run_extra.sh` builds and uses it.

With the original framework, `predict_extra_credit` compiles and reports the correct direction MPKI (for example 0.935 on jess), but a meaningless target MPKI (141.541). `measure_target` reports the real target MPKI (1.476 on jess). Fixing the getter's return type in the framework would make the two identical.

7 Discussion

- The hybrid design works because each component handles a different class of branches: the bimodal table handles strongly biased branches cheaply, and the tagged global table is used only for branches that need history (those the bimodal table mispredicts). That reduces aliasing in the global table, which is why a 1K-entry global table can beat a 32K-entry gshare.
- The hybrid is best on traces with many easy branches mixed with a few history-dependent ones (jess, perlbmk, vortex, jack, parser). gshare is slightly better on traces with many hard, history-correlated branches (mcf, twolf, vpr, gzip), where its much larger table helps. A larger or 4-way table closes that gap (5.953).
- The loop predictor adds a further gain by predicting loop exits exactly, which no counter-based predictor can do. Modelling the remaining Pentium M structures (PIR hash, iBTB) is left as future work.

8 How to run

```
cd src && make          # builds predict + predict_extra_credit
cd .. && ./run.sh traces    # required part: MPKI per trace
./run_extra.sh traces       # extra credit (correct targets)
```

(`./run` and `./run_extra` – the original csh scripts – also work if csh is installed.)